

Approach Summary - Conversational SHL Assessment Recommender

Submission for SHL AI Intern Assignment

Design Choices

- The solution is a stateless FastAPI service with GET /health and POST /chat. Each request carries the full conversation history, so the recommender reconstructs context without server-side session state. This keeps behavior predictable under evaluator replay, short time limits, and deployment restarts.
- The agent flow is simple: scope guard, clarification policy, catalog retrieval, then recommendation, refinement, or comparison. It refuses non-SHL legal/compliance advice and prompt-injection style requests.

Retrieval Setup

- The catalog is generated from public SHL Individual Test Solutions data and stored locally in data/catalog.json. URLs are normalized to the required SHL product-catalog view format, and product categories are mapped to compact test type codes such as A, K, P, B, S, and C.
- This is a lightweight RAG-style setup: the service retrieves grounded catalog records first, then builds the answer only from those records. Retrieval combines scenario rules for common hiring patterns with token scoring over product names, type labels, and generated keywords.
- I avoided a live LLM dependency in the deployed runtime so the API does not need secrets, does not fail on model/API limits, and returns consistent schema-compliant responses.

Prompt And Conversation Behavior

- If a request is vague, the service asks one clarifying question. If enough role context is available, it returns 1-10 catalog-backed SHL recommendations with name, URL, and test_type. When constraints change, it updates the prior shortlist rather than restarting. When the user asks for comparison, it compares only named assessments found in the catalog.
- end_of_conversation is true only when the user clearly confirms or locks the shortlist; otherwise it remains false so the evaluator can continue the conversation.

Evaluation Method

- I replayed the 10 public sample conversations as regression targets and added scripts/smoke_test.py for fast checks. The tests cover vague-query clarification, graduate management recommendations, full-stack Java refinement, contact center language/accent clarification, legal/HIPAA refusal, and public catalog URL formatting.
- I measured retrieval quality by checking whether expected assessments appeared in the top recommendations for known scenarios. I measured recommendation relevance by manually comparing results against the role and constraints in each trace. I measured groundedness by ensuring every recommendation URL exists in the local catalog and uses the SHL catalog format.
- I measured response accuracy and effectiveness with schema validation, manual trace review, endpoint smoke tests against the deployed Render URL, and behavior checks for refinement, comparison, refusal, and clarification.

What Did Not Work

- The current SHL products page redirects away from the older catalog index, and the rendered page no longer exposes the old product table directly. A live scraper would be brittle, so I used a normalized local catalog built from public product data instead.
- A live LLM was also not necessary for the required behavior. Deterministic retrieval gave more stable evaluation behavior and removed the need for API keys during deployment.

Deployment And Notes

- Public base URL: <https://shl-assessment-recommender-cjl9.onrender.com>
- The service is hosted on Render, so the free instance can have a cold-start delay after inactivity. GET /health and POST /chat were tested successfully on the deployed URL.
- Runtime LLM used: none. AI assistance used during implementation: OpenAI Codex/ChatGPT for scaffolding, debugging, and validation checks; final behavior was tested manually against the public traces.